

EC-231 Operating Systems

BS(CE)-2021

LAB REPORT # 9



Submitted By:

Reg. No.	Name
<u>21-CE-009</u>	<u>Huzaifa Shahbaz</u>
<u>21-CE-020</u>	<u>Muhammad Ibthesam</u>
<u>21-CE-021</u>	<u>Yawar Ali Malik</u>

Department of Computer Engineering

BS Computer Engineering Program

HITEC University Taxila

	Presentation (Format (0.5) + Title (0.5) + Conclusion(1) + Procedure(2) + Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.

Engr. Fasih Ahmad

Experiment #9

File Structure related System Calls in Linux

Objective:

In this lab we will learn about system call techniques in LINUX.

Software Used:

- 1.Virtual box
- 2.Ubuntu
- 3.Sublime text editor
- 4.GCC COMPLIER

Procedure:

System Call:

A system call is just what its name implies, a request for the operating system to do something on behalf of the users' program.

Process related system calls:

1. fork()
- 2.wait()
3. exec()
4. exit()

Basically there are total 5 types of I/O or File Structure system calls:

1. create()
2. open()
3. close()
4. read()
5. write()

Create:

Used to create a new empty file.

Syntax in C language:

```
int creat(char *filename, mode_t mode)
```

Parameter:

- 1.filename is a name of the file which you want to create
2. mode is a indicates permissions of new file.

Returns:

1. return first unused file descriptor (generally 3 when first creat use in process because 0, 1, 2 fd are reserved)
2. return -1 when error

How it work in OS:

1. Create new empty file on disk
2. Create file table entry
3. Set first unused file descriptor to point to file table entry
4. Return file descriptor used, -1 upon failure

Open:

Used to Open the file for reading, writing or both.

Syntax in C language:

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
#include <fcntl.h>
```

```
int open (const char* Path, int flags [, int mode ]);
```

Parameters:

1. Path is to file which you want to use
2. use absolute path begin with “/”, when you are not work in same directory of file.
3. Use relative path which is only file name with extension, when you are work in same directory of file.
4. flags is How you like to use
5. O_RDONLY: read only, O_WRONLY: write only, O_RDWR: read and write, O_CREAT: create file if it doesn't exist, O_EXCL: prevent creation if it already exists

How it works in OS:

1. Find existing file on disk
2. Create file table entry
3. Set first unused file descriptor to point to file table entry
4. Return file descriptor used, -1 upon failure

Close:

Tells the operating system you are done with a file descriptor and Close the file which pointed by fd.

Syntax in C language:

```
#include <fcntl.h>
```

```
int close(int fd);
```

Parameter:

fd stands for file descriptor

Return

1. 0 on success.
2. -1 on error.

How it works in the OS:

1. Destroy file table entry referenced by element fd of file descriptor table
– As long as no other process is pointing to it!
2. Set element fd of file descriptor table to NULL

Read:

From the file indicated by the file descriptor fd, the read() function reads cnt bytes of input into the memory area indicated by buf. A successful read() updates the access time for the file.

Syntax in C language:

```
size_t read (int fd, void* buf, size_t cnt);
```

Parameters:

1. file descriptor
2. buf: buffer to read data from
3. cnt: length of buffer

Returns:

How many bytes were actually read

1. return Number of bytes read on success
2. return 0 on reaching end of file
3. return -1 on error
4. return -1 on signal interrupt

Important points:

1. buf needs to point to a valid memory location with length not smaller than the specified size because of overflow.

☐ fd should be a valid file descriptor returned from open() to perform read operation because if fd is NULL then read should generate error.

☐ cnt is the requested number of bytes read, while the return value is the actual number of bytes read. Also, sometimes read system call should read less bytes than cnt.-(Use Times New Roman 12)

Tasks**Task 1:**

(a) Simulate suitable command/commands in gedit editor to open a file which should have permission of writing only. Now write five city names in the file. And add screenshot of .txt file in your lab report.

1(b). Simulate suitable command/commands in gedit editor to open a file which should have permission of reading only. Now read above written city names from the file. And show contents in the output.

CODE:

```
1  #include<stdio.h>
2  #include<fcntl.h>
3  #include<unistd.h>
4  #include<stdlib.h>
5  int main()
6  {
7      static char buffer[80];
8      char message[50];
9      printf("\n write names of cities:");
10     scanf("%50[^\t]s",message);
11     printf("%s",message);
12     int fd=open("foo.text",O_RDWR|O_CREAT);
13     if(fd!=-1)
14     {
15         printf("Name of cities are printed above\n");
16         write(fd,message,sizeof(message));
17         close(fd);
18     }
19     return 0;
20 }
```

FILE:

```
huzaihashahbaz@huzaihashahbaz-VirtualBox:~$ cd huzaihashahbaz
huzaihashahbaz@huzaihashahbaz-VirtualBox:~/huzaihashahbaz$ gcc citynames.c -o citynames
huzaihashahbaz@huzaihashahbaz-VirtualBox:~/huzaihashahbaz$ ./citynames
```

OUTPUT:

```
write names of cities:
Rawalpindi
Islamabad
Fasialabad
Sargodha
Multan

Rawalpindi
Islamabad
Fasialabad
Sargodha
Multan
```

Task 2:

Simulate suitable command/commands in gedit editor to write your introduction i.e. name, father's name, registration number and class in the file and after that

read those characters from file and save it into a variable. Display the contents of variable i.e. data read from the file.

CODE:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main() {
4      FILE *filePointer;
5      char fileName[] = "student_info.txt";
6      filePointer = fopen(fileName, "w");
7      if (filePointer == NULL) {
8          printf("Error opening file.\n");
9          return 1;
10     }
11     fprintf(filePointer, "Name: Huzaifa Shahbaz\n");
12     fprintf(filePointer, "Father's Name: Shahbaz Ghani\n");
13     fprintf(filePointer, "Registration Number: 9\n");
14     fprintf(filePointer, "Class: 3rd Year\n");
15     fclose(filePointer);
16     char data[100];
17     filePointer = fopen(fileName, "r");
18     if (filePointer == NULL) {
19         printf("Error opening file.\n");
20         return 1;
21     }
22     int i = 0;
23     while (fgets(data + i, sizeof(data) - i, filePointer) != NULL) {
24         i += strlen(data + i);
25     }
26     // Closing the file
27     fclose(filePointer);
28     printf("Data read from the file:\n%s", data);
29     return 0;
30 }
```

File:

```
huzaifashahbaz@huzaifashahbaz-VirtualBox:~$ mkdir huzaifa134
huzaifashahbaz@huzaifashahbaz-VirtualBox:~$ cd huzaifa134
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ subl studentform.c
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ gcc studentform.c -o studentform
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ ./studentform
```

OUTPUT:

```
Data read from the file:
Name: Huzaifa Shahbaz
Father's Name: Shahbaz Ghani
Registration Number: 9
Class: 3rdYear
```

Conclusion:

In conclusion, linux system calls related to file structures offer fundamental features for file and directory management. Processes can effectively create, open, read, write, and manipulate files and directories with the help of these system calls. They serve as an essential conduit between user-space programs

and the kernel, enabling smooth file operations. Linux provides a strong foundation for managing and retrieving data from the filesystem using these methods. All things considered, system calls pertaining to file structures enable programmers to create robust and adaptable Linux programs.